

Overview

Edge AI means running inference close to where data is produced (phones, cameras, gateways, embedded devices). TinyML is a subset focused on very constrained hardware, especially microcontrollers.

Why edge/TinyML matters:

- low latency for control and interaction,
- offline reliability,
- privacy by reducing raw data transfer,
- lower cloud bandwidth cost.

Constraints & Design Considerations

Resource Constraints

Typical bottlenecks:

- RAM/Flash budget,
- CPU frequency and instruction set,
- battery/power envelope,
- thermal and duty-cycle limits.

Latency and Offline Requirements

In robotics and IoT, response time can be safety-critical. Edge inference reduces round-trip delay and dependency on network availability.

Reliability and Fleet Operations

Production edge systems need:

- model/version management,
- rollback strategy,
- telemetry and drift monitoring,
- hardware-target compatibility checks.

TinyML Workflow

A practical TinyML lifecycle:

1. Collect and label sensor data (on-device or from edge-connected devices).
2. Train model in cloud/workstation.
3. Optimize model (quantization/pruning/architecture simplification).
4. Convert/export to embedded runtime format (for example TFLite Micro).
5. Integrate model into firmware/inference loop.
6. Validate on device with latency, memory, and accuracy metrics.

Quantization and Model Compression

- Post-training quantization (e.g., int8) reduces size and can speed inference.
- Pruning/distillation reduce compute footprint.

Trade-off:

- compression can degrade accuracy; must be validated against domain requirements.

Deployment Targets

Common paths include:

- microcontrollers (ultra-low-power TinyML),
- edge SoCs (higher throughput, still local inference),
- hybrid edge-cloud systems.

Example Applications

1. Wake-word detection on microcontrollers.
2. Vibration-based predictive maintenance in factories.
3. Smart home occupancy/gesture sensing.
4. Low-power environmental anomaly detection.

Business Case Studies & Exceptions

Case Study 1: Industrial Predictive Maintenance

Scenario:

- vibration/acoustic sensors monitor rotating equipment.

TinyML benefits:

- early anomaly detection near machine,
- reduced uplink data volume,
- faster alerts even with poor connectivity.

Risk:

- sensor drift and rare-failure labels can degrade model performance.

Case Study 2: Edge vs Cloud Video Analytics

Scenario:

- facility cameras require object/event detection.

Edge-first benefits:

- low latency and privacy control.

Cloud-first benefits:

- larger models and easier centralized updates.

Common compromise:

- edge for fast filtering + cloud for heavy re-analysis.

Exceptions

- If device constraints are too tight, rule-based DSP may beat TinyML.
- If updates/monitoring cannot be maintained, centralized inference may be safer operationally.

Interview Questions & Answers

1. What is TinyML?

Machine learning on resource-constrained embedded devices, typically microcontrollers.

2. **Edge AI vs cloud AI?**
Edge runs inference near data source; cloud runs centrally with higher compute but higher latency/dependency.
3. **Why quantize a model?**
To reduce model size and improve inference speed on constrained hardware.
4. **What metrics matter for TinyML deployment?**
Accuracy, RAM/Flash usage, latency, and power consumption.
5. **What is a typical TinyML workflow?**
Collect data, train, optimize, convert, deploy, validate, monitor.
6. **Why is offline capability valuable?**
It preserves functionality in unreliable or disconnected environments.
7. **When should edge inference be preferred?**
When latency, privacy, or bandwidth constraints are strict.
8. **What is a common TinyML failure mode?**
Good lab metrics but poor field robustness due to sensor/environment drift.
9. **How do you reduce on-device memory use?**
Smaller architectures, quantization, operator selection, and careful buffer planning.
10. **Why is dataset design crucial in TinyML?**
Small models are sensitive to data coverage and noise quality.
11. **How do you update models in deployed devices?**
Use controlled OTA pipelines with versioning and rollback.
12. **What is the main trade-off with aggressive compression?**
Footprint improvement vs accuracy/robustness loss.
13. **How do you test edge models before rollout?**
Hardware-in-loop benchmarks and scenario replay under realistic conditions.
14. **Can TinyML replace all edge AI workloads?**
No, high-compute tasks may require stronger edge accelerators or cloud assistance.
15. **How does TinyML relate to MLOps?**
It extends MLOps into embedded constraints: conversion, firmware integration, and fleet telemetry.

References

- Harvard TinyML professional certificate overview: <https://harvardonline.harvard.edu/program/professional-certificate-in-tiny-machine-learning-tinymml>
- TensorFlow Lite for Microcontrollers docs: <https://www.tensorflow.org/lite/microcontrollers>
- TensorFlow Lite Micro paper: <https://arxiv.org/abs/2010.08678>
- TinyML for Ubiquitous Edge AI survey: <https://arxiv.org/abs/2102.01255>
- Edge Impulse getting started workflow: <https://edgeimpulse.com/blog/getting-started-with-edge-impulse/>
- Edge Impulse MLOps platform paper: <https://arxiv.org/abs/2212.03332>
- NVIDIA Edge AI and Robotics Teaching Kit syllabus: <https://developer.nvidia.com/edge-ai-robotics-teaching-kit-syllabus>